



# PRODUTIVIDADE COM FERRAMENTAS DE IA PARA DESENVOLVEDORES

AULA BÔNUS



# O PAPEL EVOLUTIVO DA IA

- ❑ A IA está transitando de uma ferramenta de assistência para um parceiro no desenvolvimento.
- ❑ Essa relação combina a intuição e o julgamento humano com a precisão e a capacidade computacional da máquina, gerando potenciais de resultados superiores aos que poderíamos alcançar até então.



# GANHOS DE PRODUTIVIDADE MENSURÁVEIS

Fonte (McKinsey): <https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/unleashing-developer-productivity-with-generative-ai>

- ❑ Performance geral: até 2x mais rápido em tarefas comuns.
- ❑ Por tipo de tarefa:
  - ❑ Documentação: cerca de 50% do tempo.
  - ❑ Escrever código novo: quase 50% do tempo.
  - ❑ Refatoração: ~66% do tempo original.
- ❑ Complexidade alta: economia < 10%.
- ❑ Experiência do dev júnior (< 1 ano): 7% a 10% mais lento com GenAI.
- ❑ Entrega no prazo: 25% a 30% de aumento na probabilidade.
- ❑ Qualidade do código: leve melhora (menos bugs, melhor legibilidade e manutenibilidade) quando há iteração ativa com a ferramenta.
- ❑ Experiência do desenvolvedor: >2x propensão a relatar felicidade, sensação de fluxo e satisfação.
- ❑ Combinar ferramentas: usar duas (chat + IDE) melhora 1,5x a 2,5x versus usar apenas uma.



# VELOCIDADE VS. QUALIDADE

- ❑ Apesar dos ganhos de velocidade, o código gerado por IA pode introduzir vulnerabilidades críticas
- ❑ As falhas mais comuns incluem
  - ❑ Ausência de validação de entrada (levando a injeções de SQL e OS Command)
  - ❑ Falhas de autenticação e autorização
  - ❑ Uso de credenciais hard-coded
- ❑ Ou seja: a vibe do “vibe coding” só vai até a página dois...



# MUDANÇA DE HABILIDADES ESSENCIAIS

- ❑ A automação de tarefas repetitivas libera os desenvolvedores seniores para se concentrarem em atividades de maior valor, como:
  - ❑ Definição da visão e estratégia do produto
  - ❑ Priorização de funcionalidades
  - ❑ Arquitetura de sistemas complexos
  - ❑ Validação crítica do código gerado pela IA



# **TRABALHANDO O "INNER LOOP": GERAÇÃO E REFATORAÇÃO**

# ASSISTENTES DE CÓDIGO: UMA DECISÃO ESTRATÉGICA

- ❑ A escolha de um assistente de IA envolve um trade-off entre:
  - ❑ Funcionalidades
  - ❑ Integração com o ecossistema
  - ❑ Privacidade e controle de dados



**Amazon Q  
Developer**



# AMAZON Q DEVELOPER

<https://aws.amazon.com/q/developer>

- ❑ Oferece assistência "service-aware", com recomendações de recursos AWS e visibilidade em tempo real de logs e permissões diretamente no IDE. Suas capacidades "agentes" automatizam tarefas complexas como a criação de testes unitários, documentação e revisões de código.
- ❑ Prós: Integração profunda com o ecossistema AWS, automação de tarefas específicas da AWS (ex: políticas IAM), varredura de segurança e controles de acesso corporativos.
- ❑ Contras: Menos eficaz para projetos fora da AWS, suporte a linguagens mais limitado e o plano Pro possui limites de uso que podem gerar custos inesperados.





# TABNINE

<https://www.tabnine.com>

- ❑ Foco em privacidade com uma política de retenção zero de dados e a capacidade de treinar modelos customizados com o código da empresa. Oferece agentes de IA especializados para validação de requisitos, explicação de código e geração de documentação.
- ❑ Prós: Opções on-premise, personalizável com o código da equipe, amplo suporte a linguagens e modo offline.
- ❑ Contras: Pode consumir muitos recursos (CPU/memória), qualidade das sugestões pode apresentar inconsistências, plano gratuito é limitado.



# CURSOR

## CURSOR

<https://cursor.com>

- ❑ Construído como um fork do VS Code, o Cursor é um editor de código projetado em torno da IA, oferecendo uma experiência de desenvolvimento profundamente integrada. Ele se destaca por sua capacidade de analisar todo o codebase para fornecer respostas contextuais, editar código via linguagem natural e prever edições multi-linha de forma inteligente.
- ❑ Prós: Experiência de IA profundamente integrada, excelente compreensão de todo o codebase, refatoração rápida multi-arquivo e fortes opções de privacidade.
- ❑ Contras: Custo mais elevado com limites de uso que podem gerar cobranças extras, funciona como um editor próprio (sem plugin para JetBrains). Pode ter problemas de desempenho.



# JETBRAINS AI ASSISTANT

<https://www.jetbrains.com/ai-assistant>

- ❑ Ferramentas de IA podem executar refatorações complexas, como extrair lógica repetida para funções reutilizáveis, converter laços imperativos em construções funcionais, e identificar e remover código morto ou inalcançável.
- ❑ Prós: Integração nativa e profunda com os IDEs da JetBrains, sugestões de refatoração com reconhecimento de contexto e geração de testes e documentação.
- ❑ Contras: Funcionalidades principais (como autocompletar) estão atreladas aos modelos na nuvem da JetBrains, limitando o uso de modelos locais, e alguns usuários relatam latência.

# **TRABALHANDO O "OUTER LOOP": QUALIDADE E TESTES**





# SONAR: AI CODE ASSURANCE

<https://www.sonarsource.com/solutions/ai/ai-code-assurance>

- ❑ O SonarQube evoluiu para além da análise tradicional com o "AI Code Assurance", um workflow para validar código gerado por IA. Ele utiliza um Quality Gate específico que impõe padrões mais rígidos em código novo, como cobertura de testes de 80% e duplicação máxima de 3%, garantindo que a velocidade da IA não comprometa a qualidade.
- ❑ Prós: Garante a qualidade do código gerado por IA, oferece correções automáticas com AI CodeFix e se integra a pipelines de CI/CD para bloquear merges que não atendem aos padrões.
- ❑ Contras: A configuração inicial pode ser complexa, pode gerar falsos positivos que exigem ajuste fino e as funcionalidades de IA mais avançadas são parte das edições pagas.

# SNYK POWERED BY DEEPCODE AI

<https://snyk.io/platform/deepcode-ai/>



- ❑ O Snyk Code utiliza o motor "DeepCode AI", treinado em milhões de repositórios para entender o fluxo de dados e o contexto do código. Isso permite detectar vulnerabilidades complexas em código gerado por IA e humano. O "Snyk Agent Fix" oferece correções automáticas com um clique, diretamente no IDE.
- ❑ Prós: Detecção precisa com baixa taxa de falsos positivos, correções automáticas que aceleram a remediação e integração profunda no IDE para feedback em tempo real.
- ❑ Contras: O preço pode ser elevado para equipes maiores, a interface pode ser complexa para gerenciar um grande volume de vulnerabilidades e o foco principal é segurança, com menos ênfase em outros aspectos de qualidade de código.

The logo for diffblue. is displayed in a white, lowercase, monospaced font within a dark blue rectangular box. The background of the slide features a dark blue abstract shape on the left and a light blue circle on the right.

## DIFFBLUE COVER

<https://www.diffblue.com>

- ❑ O aprendizado por reforço (RL) treina um modelo para otimizar métricas de qualidade de teste. Um "Reward Model" pontua os testes gerados com base em análises estáticas, ensinando o agente a evitar "test smells" (anti-padrões) e a produzir testes sintaticamente corretos.
- ❑ Prós: Gera testes garantidos para compilar e rodar, opera de forma autônoma, ideal para código legado e mantém o código-fonte privado (on-premise).
- ❑ Contras: Focado exclusivamente em Java, pode ter uma curva de aprendizado para otimização e o custo de licenciamento pode ser um fator limitante.



# MINTLIFY

<https://mintlify.com>

- ❑ Plataforma de documentação "AI-native" projetada para ser consumida tanto por humanos quanto por LLMs, utilizando IA em todo o ciclo de vida. Ela oferece um escritor com consciência de contexto para auxiliar na criação e manutenção do conteúdo, e um assistente de IA que transforma a documentação em uma experiência de conversação para os usuários finais.
- ❑ Prós: Experiência de IA completa, incluindo um assistente para que os usuários possam "conversar" com a documentação; geração automática de páginas a partir de especificações OpenAPI e um playground de API interativo; e um fluxo de trabalho colaborativo que suporta tanto desenvolvedores (via Git) quanto equipes não-técnicas (editor visual).
- ❑ Contras: O foco principal é na criação de documentação de alta qualidade para o usuário final (guias, portais de API), sendo menos especializado em manter documentação interna sincronizada com trechos de código específicos em tempo real.

**OU SEJA...**



“

*A IA não substitui o desenvolvedor, mas o desenvolvedor que domina a IA certamente terá uma vantagem competitiva. A integração inteligente dessas ferramentas é a chave para a próxima geração de produtividade para desenvolvedores.*

”

**ATÉ A PRÓXIMA!**